# LandVerifyCM

## Complete Team Project Summary

Tamaffo Fabrice — Project Leader & Database Manager | Nkam Titcha — Frontend & Mobile Developer | Kemgang Prince — Backend Developer

PART 1

# Tamaffo Fabrice

**Project Opening & Team Leadership**

### Opening statement

Good morning. Land is one of the most important investments a family can make. However, forged titles, duplicated records, unclear ownership histories, and overlapping parcel locations can expose people in Cameroon to financial loss and long legal disputes. Our project, LandVerifyCM, provides a practical digital first check before a land transaction.

LandVerifyCM is a bilingual web and mobile platform that connects citizens, professionals, and authorized administrators to one land-information service. A user can search a title, review the verification status, examine relevant parcel information, and identify warning signs earlier. Administrators maintain the registry through controlled and traceable workflows.

### Team organization

As project leader, I coordinated three connected areas: Nkam Titcha developed the web and mobile user experience; Kemgang Prince developed the backend services and verification logic; and I managed the database design, data integrity, and project coordination. The three layers share one structure so that information displayed to users matches what is processed and stored.

In my opening, I will first describe the problem in everyday language, then introduce the platform without going too deeply into code. I will explain that the purpose is to help a buyer notice risks earlier, not to declare legal ownership independently of the authorities. I will then introduce the three team roles and show how the user interface, backend rules, and database depend on one another.

PART 2

# Nkam Titcha

**Frontend & Mobile Development**

## My responsibility

As Frontend Developer, I built the interaction layer through which citizens and administrators use LandVerifyCM. My responsibility covered both the React website and the Expo/React Native mobile client. I converted API responses into readable screens, managed navigation and form states, protected role-specific pages, and kept the experience consistent in English and French.

## Web frontend

The website was developed with React and TypeScript. React made it possible to divide the interface into reusable components such as the navigation bar, land cards, status badges, maps, protected routes, and form controls. TypeScript defined the expected shape of users, parcels, documents, and ownership records, helping us catch interface errors during development.

Vite was used because it provides a fast development server and efficient production builds. Tailwind CSS supplied responsive styling for phones, tablets, and desktop computers. React Router organizes pages such as Home, Search, Browse, Land Details, Login, Profile, and the administrator dashboard. Context providers keep authentication and the English/French language selection available throughout the application.

## Main user experience

Users can enter a land title, browse records by quarter, and view a clear verification result. The interface presents loading, empty, success, and error states so the user always understands what is happening. Land details can include area, land-use type, approval year, ownership history, supporting documents, and GPS information according to the user's access level.

React Leaflet and OpenStreetMap convert coordinates into an interactive map. The web map supports street and satellite views and allows coordinates to be copied. Administrators receive protected screens for uploading, editing, deactivating, and reviewing land records, ownership entries, and PDF or image evidence.

## Mobile application

The mobile client was built with Expo and React Native so one codebase can serve Android, iOS, and a browser build. Users can search and browse parcels, inspect status and ownership information, view an embedded map, and open a parcel in Google Maps. The mobile administrator screens also support record management, document selection, user review, and audit logs.

Expo SecureStore protects JWT session data on native devices. Expo DocumentPicker, FileSystem, and Sharing support evidence selection, downloading, and sharing. Friendly error messages explain expired sessions, permission problems, server errors, or connection failures in plain language.

## How the interface communicates with the system

On the website, an Axios service stores the backend address and automatically attaches the logged-in user's Bearer token. If the API returns an unauthorized response, the application clears the expired session and returns the user to Login. The mobile client uses a typed fetch service with the same API routes. It recognizes invalid credentials, forbidden actions, missing records, rate limits, server errors, and network failures, then converts them into messages that ordinary users can understand.

A useful example to present is the search journey. The user enters a title number; the screen sends the query to the land-search endpoint; a loading state appears; and the response is converted into a land card and status badge. Selecting the record opens its details. This demonstrates that my work was not only choosing colours: it included data flow, component state, navigation, access control, validation, error recovery, and readable presentation of verification evidence.

## Why this helps Cameroon

Many people access online services mainly through phones. Providing both browser and mobile access makes early land checks more practical. Bilingual screens improve accessibility, while clear status labels, maps, ownership information, and evidence help users ask better questions before paying. The interface supports due diligence and discussion with professionals; it does not replace the formal decision of Cameroon’s competent land authorities.

During my presentation, I will demonstrate the interface in this order: Home, Search, Result, Land Details, Map, language switch, and finally the protected administrator screens. This sequence lets the audience see the citizen experience before the more technical management functions.

PART 3

# Kemgang Prince

**Backend Development**

## My responsibility

As Backend Developer, I built the service that connects the web and mobile interfaces to the PostgreSQL database. The backend receives requests, validates data, applies business rules, controls access, performs verification, and returns consistent JSON responses to both clients.

## API architecture and features

The backend uses Node.js, Express, and TypeScript. Node.js and Express are suitable for a lightweight REST API, while TypeScript reduces mistakes in request data and shared objects. Routes are separated into authentication, public land, and administrator groups. Controllers then handle registration, login, password recovery, land search, quarter browsing, parcel details, uploads, updates, deactivation, ownership records, users, and audit history.

A normal request follows a clear flow: the web or mobile client sends an HTTPS request; an Express route selects the endpoint; middleware checks authentication and authorization; a controller applies the rule; and Prisma reads or writes PostgreSQL data. Pagination limits large result sets, and selected database fields prevent password hashes from being exposed.

## Automated land verification

The verification service checks every active parcel when an administrator creates or updates a record. First, a matching title number is classified as Duplicate. Next, the Haversine formula calculates the real-world distance between GPS coordinate pairs using the Earth's radius. A parcel within 10 metres of another active parcel is Duplicate; a parcel within 50 metres is Suspicious and requires manual review; otherwise, it is Valid.

During an update, the current parcel is excluded from its own comparison. A duplicate upload is blocked with an HTTP 409 response, while a suspicious record can be stored with an explanation. Keeping this logic in one backend service ensures that the website and mobile app receive the same evidence-based result.

## Example of a backend operation

When an administrator uploads a parcel, Multer first receives the supporting files and Zod checks the text and numeric fields. The controller asks the verification service to compare the proposed title and GPS point with active parcels. If the result is acceptable, Prisma creates the parcel, its document references, and its ownership entries. The controller then writes an audit event and sends the completed record back to the client. If a duplicate is found, the process stops before storing the parcel and returns a conflict response with the reason.

This example is important in my presentation because it shows that the backend is more than a set of URLs. It coordinates validation, security, verification, database operations, files, and errors as one transaction-like workflow. I will also explain that updates rerun verification, deactivation preserves history, and public list endpoints use pagination so the service remains practical as the registry grows.

## Security and supporting technologies

Zod validates emails, passwords, areas, coordinates, years, and ownership data. bcrypt hashes passwords with 12 rounds, and JWT tokens identify the user and role. Bearer authentication and an

ADMIN role check protect management functions. Rate limiting reduces repeated login and password-reset attempts; Helmet, CORS rules, compression, and request-size limits strengthen the public API.

Multer controls document uploads and accepts up to five PDF, JPG, or PNG files of 10 MB each. Important create, update, deactivation, and ownership actions receive timestamped audit records. Prisma was selected because it provides typed queries and a safe connection between TypeScript and the relational database.

## Why this helps Cameroon

The backend turns land information into checks that people can act on. Buyers can identify a duplicate title or nearby GPS conflict before committing money. Families and professionals can follow ownership evidence, while administrators maintain records through controlled, traceable actions. The result is one reliable service for both the browser and mobile application.

I will present my part by following one request from the phone to the database and back. I will then show the 10-metre and 50-metre decision rules, followed by the security controls. This order makes the backend understandable even to an audience that does not write code.

PART 4

# Tamaffo Fabrice

**Database Management, Cameroon Impact & Project Conclusion**

## My database and leadership responsibility

As Database Manager, I designed the relational structure in PostgreSQL through Prisma, prepared migrations and demonstration seed data, and connected the stored information to verification and administration. As Project Leader, I also coordinated the shared requirements and ensured that the frontend, backend, and database represented the same users, parcels, statuses, and ownership records.

## Database design

The User table stores the account name, unique email, password hash, ADMIN or USER role, active state, and timestamps. LandParcel stores the unique title number, owner, quarter, area, GPS latitude and longitude, verification status, approval year, land-use type, notes, active state, and the administrator who uploaded it.

OwnershipRecord preserves the chronological ownership chain, including original ownership, purchase, inheritance, donation, or court order. LandDocument links supporting file metadata to the correct parcel. AuditLog records the responsible user, action, changed information, parcel, and time. These relationships keep evidence connected instead of storing isolated files or names.

## Technologies and data reliability

PostgreSQL was chosen for strong relational constraints, joins, transactions, and durable structured storage. Prisma defines the schema, relationships, indexes, and migrations while giving the TypeScript backend type-safe queries. SQL migrations make database changes repeatable across development and deployment, and seed data provides consistent records for demonstrations.

UUID primary keys avoid predictable identifiers. A unique constraint prevents two stored parcels from sharing a title number. Foreign keys keep users, parcels, documents, ownership records, and audit events linked correctly. Indexes improve searches by quarter, verification status, active state, and related records. Deactivation preserves history instead of silently deleting an important land record.

## How I managed the data flow and the team

A complete record begins when an administrator submits parcel details, ownership history, GPS coordinates, and evidence. The backend validates and verifies the submission, after which Prisma writes connected rows in PostgreSQL. Later searches read the same source of truth, and audit records show which administrator performed a sensitive action. I will use this flow to explain why database design affects every screen and every verification result.

My project-lead work included keeping the team focused on the same functional requirements, agreeing on field names and status values, reviewing how each layer exchanged data, and organizing the work so that changes could move from feature development through integration and release preparation. This coordination was necessary because a correct database field is only useful when the backend returns it consistently and the frontend explains it clearly.

## Conclusion

To conclude, LandVerifyCM applies software engineering to a real concern in Cameroon. Nkam's interfaces make verification clear and accessible, Prince's backend applies consistent security and fraud-detection rules, and the database preserves the evidence and history behind every result.

For Cameroonians, this can mean fewer blind decisions, earlier warnings, clearer ownership information, and stronger questions before committing hard-earned money. LandVerifyCM is not a replacement for government land authorities or legal professionals; it is a transparent first check that supports safer due diligence. Thank you.

For the final presentation, I will summarize the contribution of each team member, return to the original problem of land fraud and uncertainty, state the responsible limits of the platform, and finish with the practical value: earlier warnings, traceable records, and better-informed decisions.